

crop.py

This handles **Sub-task (a): Crop classes and inheritance.**

```
class Crop:
```

```
    """
```

```
    Base Crop class.
```

```
    Contains common condition checking functionality.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        name,
```

```
        soil_moisture_min,
```

```
        temperature_min,
```

```
        temperature_max,
```

```
        humidity_min
```

```
    ):
```

```
        self.name = name
```

```
        self.soil_moisture_min = soil_moisture_min
```

```
        self.temperature_min = temperature_min
```

```
        self.temperature_max = temperature_max
```

```
        self.humidity_min = humidity_min
```

```
    def evaluate_conditions(
```

```
        self,
```

```
        soil_moisture,
```

```
        temperature,
```

```
        humidity
```

```
):  
  
    alerts = []  
  
    # Check soil moisture  
    if soil_moisture < self.soil_moisture_min:  
        alerts.append(  
            "Low Soil Moisture - Irrigation Required"  
        )  
  
    # Check temperature  
    if temperature < self.temperature_min:  
        alerts.append(  
            "Low Temperature"  
        )  
  
    if temperature > self.temperature_max:  
        alerts.append(  
            "High Temperature"  
        )  
  
    # Check humidity  
    if humidity < self.humidity_min:  
        alerts.append(  
            "Low Humidity"  
        )  
  
    return alerts
```

```
class CerealCrop(Crop):
    """
    Child class for cereal crops.
    """

    def __init__(self, thresholds):
        super().__init__(
            "Cereal",
            thresholds["soil_moisture_min"],
            thresholds["temperature_min"],
            thresholds["temperature_max"],
            thresholds["humidity_min"]
        )
```

```
class VegetableCrop(Crop):
    """
    Child class for vegetable crops.
    """

    def __init__(self, thresholds):
        super().__init__(
            "Vegetable",
            thresholds["soil_moisture_min"],
            thresholds["temperature_min"],
```

```
        thresholds["temperature_max"],
        thresholds["humidity_min"]
    )
```

```
class FruitCrop(Crop):
```

```
    """
```

```
    Child class for fruit crops.
```

```
    """
```

```
    def __init__(self, thresholds):
```

```
        super().__init__(
```

```
            "Fruit",
```

```
            thresholds["soil_moisture_min"],
```

```
            thresholds["temperature_min"],
```

```
            thresholds["temperature_max"],
```

```
            thresholds["humidity_min"]
```

```
        )
```

3. data_reader.py

This handles **Sub-task (b): Reading CSV sensor data.**

```
import csv
```

```
import logging
```

```
def read_sensor_data(filename):
```

```
data = []
```

```
try:
```

```
    with open(filename, "r", newline="") as file:
```

```
        reader = csv.DictReader(file)
```

```
        required_columns = [
```

```
            "timestamp",
```

```
            "soil_moisture",
```

```
            "temperature",
```

```
            "humidity",
```

```
            "crop_type",
```

```
            "irrigation_level"
```

```
        ]
```

```
        # Check columns
```

```
        for column in required_columns:
```

```
            if column not in reader.fieldnames:
```

```
                raise ValueError(
```

```
                    f"Missing column: {column}"
```

```
                )
```

```
        # Read each row
```

```
        for row_number, row in enumerate(reader, start=2):
```

try:

```
    row["soil_moisture"] = float(
        row["soil_moisture"]
    )
```

```
    row["temperature"] = float(
        row["temperature"]
    )
```

```
    row["humidity"] = float(
        row["humidity"]
    )
```

```
    row["irrigation_level"] = float(
        row["irrigation_level"]
    )
```

```
    data.append(row)
```

except ValueError:

```
    print(
        f"Invalid data in CSV row {row_number}"
    )
```

```
        logging.warning(  
            f"Invalid data in CSV row {row_number}"  
        )
```

```
logging.info(  
    f"Successfully loaded {len(data)} sensor records"  
)
```

```
return data
```

```
except FileNotFoundError:
```

```
    print("Error: sensor_data.csv not found.")
```

```
    logging.error(  
        "sensor_data.csv file not found"  
    )
```

```
    return []
```

```
except Exception as e:
```

```
    print("Error reading sensor data:", e)
```

```
    logging.error(  
        f"Error reading sensor data: {e}"  
    )
```

```
return []
```

4. analyzer.py

This is the main **analysis module**.

It loads the appropriate crop class and compares sensor values with JSON thresholds.

```
from crop import (
```

```
    CerealCrop,
```

```
    VegetableCrop,
```

```
    FruitCrop
```

```
)
```

```
import logging
```

```
def create_crop(crop_type, config):
```

```
    crop_type = crop_type.lower().strip()
```

```
    if crop_type == "cereal":
```

```
        return CerealCrop(
```

```
            config["cereal"]
```

```
        )
```

```
    elif crop_type == "vegetable":
```



```
    return VegetableCrop(  
        config["vegetable"]  
    )
```

```
elif crop_type == "fruit":
```

```
    return FruitCrop(  
        config["fruit"]  
    )
```

```
else:
```

```
    return None
```

```
def analyze_data(sensor_data, config):
```

```
    results = []
```

```
    for row in sensor_data:
```

```
        crop_type = row["crop_type"]
```

```
        crop = create_crop(  
            crop_type,  
            config  
        )
```

if crop is None:

```
    logging.warning(  
        f"Unknown crop type: {crop_type}"  
    )
```

continue

Evaluate crop conditions

```
alerts = crop.evaluate_conditions(  
    row["soil_moisture"],  
    row["temperature"],  
    row["humidity"]  
)
```

Determine crop status

```
if len(alerts) == 0:  
    status = "Healthy"
```

```
elif len(alerts) <= 2:  
    status = "Warning"
```

```
else:  
    status = "Critical"
```

```
result = {
```

```
        "timestamp": row["timestamp"],

        "crop_type": row["crop_type"],

        "soil_moisture":
            row["soil_moisture"],

        "temperature":
            row["temperature"],

        "humidity":
            row["humidity"],

        "irrigation_level":
            row["irrigation_level"],

        "alerts": alerts,

        "status": status
    }

    results.append(result)

logging.info(
    "Sensor data analysis completed"
)
```

```
return results
```

5. alert_manager.py

This handles **Sub-task (d): Alert generation**.

```
import os
```

```
import logging
```

```
def generate_alerts(results):
```

```
    os.makedirs("output", exist_ok=True)
```

```
    filename = "output/alerts.txt"
```

```
    alert_count = 0
```

```
    with open(filename, "w") as file:
```

```
        file.write("=" * 70 + "\n")
```

```
        file.write("SMART AGRICULTURE ALERTS\n")
```

```
        file.write("=" * 70 + "\n\n")
```

```
        for result in results:
```

```
            if result["alerts"]:
```

```

for alert in result["alerts"]:

    message = (

        f"{result['timestamp']} | "

        f"Crop: {result['crop_type']} | "

        f"Soil Moisture: "

        f"{result['soil_moisture']}% | "

        f"Temperature: "

        f"{result['temperature']}°C | "

        f"Humidity: "

        f"{result['humidity']}% | "

        f"ALERT: {alert}"

    )

    print("⚠️", message)

    file.write(

        message + "\n"

    )

    alert_count += 1

if alert_count == 0:

    file.write(

        "No alerts generated. "

        "All crops are in healthy condition.\n"

```

```
)

logging.info(
    f"Generated {alert_count} alerts"
)

return alert_count
```

6. report.py

This handles **Sub-task (f): Daily report.**

```
import os

from collections import Counter

import logging


def generate_report(results):

    os.makedirs("output", exist_ok=True)

    filename = "output/daily_report.txt"

    total_records = len(results)

    healthy_count = 0
    warning_count = 0
    critical_count = 0
```

```
crop_counter = Counter()
```

```
alert_counter = Counter()
```

```
total_irrigation = 0
```

```
# Analyze results
```

```
for result in results:
```

```
    crop_counter[  
        result["crop_type"]  
    ] += 1
```

```
# Status count
```

```
if result["status"] == "Healthy":
```

```
    healthy_count += 1
```

```
elif result["status"] == "Warning":
```

```
    warning_count += 1
```

```
elif result["status"] == "Critical":
```

```
    critical_count += 1
```

```
# Irrigation
```

```
total_irrigation += result[
```

```

        "irrigation_level"
    ]

    # Alerts
    for alert in result["alerts"]:

        alert_counter[alert] += 1

    # Calculate average irrigation
    if total_records > 0:

        average_irrigation = (
            total_irrigation / total_records
        )

    else:

        average_irrigation = 0

    # Generate report
    with open(filename, "w") as file:

        file.write("=" * 70 + "\n")

        file.write(
            "SMART AGRICULTURE DAILY CROP REPORT\n"
        )

        file.write("=" * 70 + "\n\n")

```



```
file.write(  
    f"Total Sensor Records: "  
    f"{total_records}\n"  
)
```

```
file.write(  
    f"Healthy Records: "  
    f"{healthy_count}\n"  
)
```

```
file.write(  
    f"Warning Records: "  
    f"{warning_count}\n"  
)
```

```
file.write(  
    f"Critical Records: "  
    f"{critical_count}\n"  
)
```

```
file.write(  
    f"Average Irrigation Level: "  
    f"{average_irrigation:.2f}%\n\n"  
)
```

```
# Crop summary
```

```
file.write(  
    "CROP SUMMARY\n"  
)
```

```
file.write("-" * 40 + "\n")
```

```
for crop, count in crop_counter.items():
```

```
    file.write(  
        f"{crop}: {count} records\n"  
    )
```

```
file.write("\n")
```

```
# Alert summary
```

```
file.write(  
    "ALERT SUMMARY\n"  
)
```

```
file.write("-" * 40 + "\n")
```

```
if alert_counter:
```

```
    for alert, count in alert_counter.items():
```

```
        file.write(  
            f"{alert}: {count}\n"
```

```
)

else:

    file.write(
        "No alerts generated.\n"
    )

    file.write("\n")

    file.write(
        "RESOURCE USAGE\n"
    )

    file.write("-" * 40 + "\n")

    file.write(
        f"Average irrigation level: "
        f"{average_irrigation:.2f}%\n"
    )

logging.info(
    "Daily report generated successfully"
)

return filename
```

7. config.json

This is **Sub-task (c)**.

The question says thresholds must be loaded from a JSON configuration file.

```
{  
  "cereal": {  
    "soil_moisture_min": 40,  
    "temperature_min": 18,  
    "temperature_max": 35,  
    "humidity_min": 45  
  },  
  
  "vegetable": {  
    "soil_moisture_min": 50,  
    "temperature_min": 15,  
    "temperature_max": 30,  
    "humidity_min": 50  
  },  
  
  "fruit": {  
    "soil_moisture_min": 45,  
    "temperature_min": 18,  
    "temperature_max": 32,  
    "humidity_min": 50  
  }  
}
```

8. sensor_data.csv

Create this file:

```
timestamp,soil_moisture,temperature,humidity,crop_type,irrigation_level
```

```
2026-08-24 08:00,35,30,60,Cereal,20
```

```
2026-08-24 09:00,55,25,55,Vegetable,45
```

```
2026-08-24 10:00,42,34,45,Fruit,35
```

```
2026-08-24 11:00,30,38,40,Cereal,15
```

```
2026-08-24 12:00,60,28,65,Vegetable,50
```

```
2026-08-24 13:00,40,20,45,Fruit,30
```

```
2026-08-24 14:00,48,27,60,Cereal,40
```

```
2026-08-24 15:00,35,29,55,Vegetable,25
```

```
2026-08-24 16:00,55,31,48,Fruit,50
```

```
2026-08-24 17:00,25,36,35,Cereal,10
```

9. main.py

This is the **main program** that connects everything.

```
import json
```

```
import logging
```

```
import os
```

```
from data_reader import read_sensor_data
```

```
from analyzer import analyze_data
```

```
from alert_manager import generate_alerts
```

```
from report import generate_report
```

```
# -----
```

```
# CREATE OUTPUT DIRECTORY
```

```
# -----
```

```
os.makedirs("output", exist_ok=True)
```

```
# -----
```

```
# LOGGING CONFIGURATION
```

```
# -----
```

```
logging.basicConfig(  
    filename="output/agriculture.log",  
    level=logging.INFO,  
    format="%(asctime)s - %(levelname)s - %(message)s"  
)
```

```
# -----
```

```
# LOAD JSON CONFIGURATION
```

```
# -----
```

```
def load_config(filename):
```

```
    try:
```

```
        with open(filename, "r") as file:
```

```
            config = json.load(file)
```

```
logging.info(  
    "Configuration file loaded successfully"  
)
```

```
return config
```

```
except FileNotFoundError:
```

```
print(  
    "ERROR: config.json file not found."  
)
```

```
logging.error(  
    "config.json file not found"  
)
```

```
return None
```

```
except json.JSONDecodeError:
```

```
print(  
    "ERROR: Invalid JSON configuration."  
)
```

```
logging.error(  
    "Invalid JSON configuration"
```

```
)
```

```
return None
```

```
# -----
```

```
# DISPLAY SENSOR DATA
```

```
# -----
```

```
def display_results(results):
```

```
    print("\n")
```

```
    print("=" * 90)
```

```
    print("CROP CONDITION ANALYSIS")
```

```
    print("=" * 90)
```

```
    for result in results:
```

```
        print(
```

```
            f"\nTimestamp    : "
```

```
            f"{result['timestamp']}"
```

```
        )
```

```
        print(
```

```
            f"Crop Type      : "
```

```
            f"{result['crop_type']}"
```

```
        )
```



```
print(  
    f"Soil Moisture : "  
    f"{result['soil_moisture']}%"  
)
```

```
print(  
    f"Temperature  : "  
    f"{result['temperature']}°C"  
)
```

```
print(  
    f"Humidity    : "  
    f"{result['humidity']}%"  
)
```

```
print(  
    f"Irrigation   : "  
    f"{result['irrigation_level']}%"  
)
```

```
print(  
    f"Status       : "  
    f"{result['status']}"  
)
```

```
if result["alerts"]:
```

```
print("Alerts:")
```

```
for alert in result["alerts"]:
```

```
    print(  
        f" △ {alert}"  
    )
```

```
else:
```

```
    print(  
        "Alerts      : None"  
    )
```

```
# -----
```

```
# MAIN FUNCTION
```

```
# -----
```

```
def main():
```

```
    print("\n")
```

```
    print("=" * 70)
```

```
    print(  
        "    SMART AGRICULTURE CROP MONITORING SYSTEM"  
    )
```

```
print("=" * 70)
```

```
logging.info(  
    "Smart Agriculture System started"  
)
```

```
# -----
```

```
# STEP 1: LOAD CONFIGURATION
```

```
# -----
```

```
print("\n[1] Loading crop configuration...")
```

```
config = load_config("config.json")
```

```
if config is None:
```

```
    return
```

```
print(  
    "✓ Configuration loaded successfully"  
)
```

```
# -----
```

```
# STEP 2: READ SENSOR DATA
```

```
# -----
```

```
print("\n[2] Reading sensor data...")
```

```

sensor_data = read_sensor_data(
    "sensor_data.csv"
)

if not sensor_data:

    print(
        "No sensor data available."
    )

    return

print(
    f"✓ {len(sensor_data)} sensor records loaded"
)

# -----
# STEP 3: ANALYZE SENSOR DATA
# -----

print(
    "\n[3] Analyzing crop conditions..."
)

results = analyze_data(
    sensor_data,

```

```
    config
)

print(
    "✓ Sensor analysis completed"
)

# -----
# STEP 4: DISPLAY RESULTS
# -----

display_results(results)

# -----
# STEP 5: GENERATE ALERTS
# -----

print("\n")
print(
    "[4] Generating alerts..."
)

alert_count = generate_alerts(
    results
)

print(
```

```
        f"✓ {alert_count} alerts generated"
    )

# -----

# STEP 6: GENERATE REPORT

# -----

print(
    "\n[5] Generating daily report..."
)

report_file = generate_report(
    results
)

print(
    f"✓ Report generated: {report_file}"
)

# -----

# FINAL MESSAGE

# -----

logging.info(
    "Smart Agriculture System completed successfully"
)
```

```

print("\n")
print("=" * 70)
print("          SYSTEM EXECUTION COMPLETED")
print("=" * 70)

print("\nGenerated Files:")
print("✓ output/alerts.txt")
print("✓ output/agriculture.log")
print("✓ output/daily_report.txt")

print("\nThank you!")

```

```

# -----
# PROGRAM ENTRY POINT
# -----

```

```

if __name__ == "__main__":

```

```

    main()

```

10. Run It in VS Code

Open the terminal inside your project folder:

```
python main.py
```

You should see something like:

```

=====

SMART AGRICULTURE CROP MONITORING SYSTEM

```

=====

[1] Loading crop configuration...

✓ Configuration loaded successfully

[2] Reading sensor data...

✓ 10 sensor records loaded

[3] Analyzing crop conditions...

✓ Sensor analysis completed

Then:

=====

=====

CROP CONDITION ANALYSIS

=====

=====

Timestamp : 2026-08-24 08:00

Crop Type : Cereal

Soil Moisture : 35.0%

Temperature : 30.0°C

Humidity : 60.0%

Irrigation : 20.0%

Status : Warning

Alerts:

⚠ Low Soil Moisture - Irrigation Required

And for an unhealthy reading:

Timestamp : 2026-08-24 11:00

Crop Type : Cereal
Soil Moisture : 30.0%
Temperature : 38.0°C
Humidity : 40.0%
Irrigation : 15.0%
Status : Critical

Alerts:

- ⚠ Low Soil Moisture - Irrigation Required
- ⚠ High Temperature
- ⚠ Low Humidity

11. Output Files

After execution, your folder will contain:

alerts.txt

Example:

```
=====
SMART AGRICULTURE ALERTS
=====
```

```
2026-08-24 08:00 | Crop: Cereal | Soil Moisture: 35.0% | Temperature: 30.0°C | Humidity: 60.0%
| ALERT: Low Soil Moisture - Irrigation Required
```

```
2026-08-24 11:00 | Crop: Cereal | Soil Moisture: 30.0% | Temperature: 38.0°C | Humidity: 40.0%
| ALERT: Low Soil Moisture - Irrigation Required
```

```
2026-08-24 11:00 | Crop: Cereal | Soil Moisture: 30.0% | Temperature: 38.0°C | Humidity: 40.0%
| ALERT: High Temperature
```

agriculture.log

Example:

2026-08-24 22:45:10 - INFO - Smart Agriculture System started
2026-08-24 22:45:10 - INFO - Configuration file loaded successfully
2026-08-24 22:45:10 - INFO - Successfully loaded 10 sensor records
2026-08-24 22:45:10 - INFO - Sensor data analysis completed
2026-08-24 22:45:10 - INFO - Generated 10 alerts
2026-08-24 22:45:10 - INFO - Daily report generated successfully

daily_report.txt

Example:

```
=====
SMART AGRICULTURE DAILY CROP REPORT
=====
```

Total Sensor Records: 10
Healthy Records: 3
Warning Records: 5
Critical Records: 2
Average Irrigation Level: 32.00%

CROP SUMMARY

Cereal: 4 records
Vegetable: 3 records
Fruit: 3 records

ALERT SUMMARY

Low Soil Moisture - Irrigation Required: 5

High Temperature: 2

Low Humidity: 3

RESOURCE USAGE

Average irrigation level: 32.00%